

Masr Delivery — Java Capstone Project

Part A — The Business Domain

A.1 Restaurants

The platform carries restaurants. Every restaurant has an identifier that is unique platform-wide, a display name, a district it operates from (for example Maadi, Dokki, Faisal, Nasr City, Heliopolis), one or more cuisine categories, an average rating between 0.0 and 5.0, and an open/closed status.

Each restaurant offers menu items. A menu item has an identifier unique within its restaurant, a name, a price in EGP, a category, a preparation time in minutes, and an availability flag.

Menu items are not all the same kind of thing, and they must not be priced the same way. The platform supports at least three kinds:

- A **standard item** is sold at its listed price.
- A **combo** bundles several items together and is sold at a discount off the sum of its parts.
- A **weighted item** (grilled meats, for example) is priced per kilogram, and the customer orders a quantity in kilograms rather than a count.

A new kind of item will certainly be added after launch. Adding it must not require editing the code that prices an order.

A.2 Customers

A customer has a unique identifier, a name, an Egyptian mobile number, one or more saved addresses (each with a district and a free-text detail line), a wallet balance in EGP, and a loyalty tier.

There are three loyalty tiers, and a customer's tier is **derived from their completed order count**, never set by hand:

Tier	Completed orders	Benefit
Bronze	0 – 9	none
Silver	10 – 29	10% off the delivery fee
Gold	30 or more	delivery fee waived entirely

A.3 Riders

A rider has a unique identifier, a name, a vehicle type (motorcycle, bicycle, or car), a current district, an availability status, and a running count of completed deliveries.

A rider may hold **at most one active order at a time**. This rule is absolute and is the single most important correctness property of your system.

A.4 Orders

An order links one customer, one restaurant, one delivery address, and one or more line items (each a menu item plus a quantity). It records the moment it was placed, and it moves through a lifecycle:

```
PLACED → ACCEPTED → PREPARING → READY → ASSIGNED → OUT_FOR_DELIVERY → DELIVERED
```

At any point before OUT_FOR_DELIVERY, an order may instead move to CANCELLED.

Illegal transitions must be rejected, not silently ignored. An order cannot jump from PLACED to DELIVERED. A DELIVERED order cannot be cancelled. A CANCELLED order is final.

Part B — Pricing Rules

Every order total must be computed by these rules, in this order:

1. **Subtotal** — the sum of every line item. Standard items: price × count. Combos: the bundle price × count. Weighted items: price per kg × kilograms ordered.
2. **Delivery fee** — 15 EGP base, plus 3 EGP for every kilometre beyond the first 3 km between the restaurant's district and the delivery district. Then the loyalty benefit from A.2 is applied.
3. **Service fee** — 10% of the subtotal, rounded to the nearest piastre.
4. **Promotion** — at most one promotion code per order, applied to the subtotal only (never to fees).
5. **Total** — subtotal + delivery fee + service fee – promotion discount. A total may never be negative.

The platform supports several promotion types and will add more: a percentage off the subtotal with a maximum discount cap, a fixed amount off, and free delivery. A promotion may carry conditions — a minimum subtotal, an expiry date, a restriction to one district, or a restriction to first-time customers.

Worked example. A customer in Faisal orders from a restaurant in Maadi, 12 km away. Subtotal is 240.00 EGP. The customer is Bronze. Promotion NILE20 gives 20% off the subtotal, capped at 50 EGP.

- Delivery fee: $15 + (12 - 3) \times 3 = \mathbf{42.00 \text{ EGP}}$
- Service fee: $240.00 \times 0.10 = \mathbf{24.00 \text{ EGP}}$
- Promotion: $240.00 \times 0.20 = 48.00$, which is under the 50.00 cap, so **48.00 EGP**
- Total: $240.00 + 42.00 + 24.00 - 48.00 = \mathbf{258.00 \text{ EGP}}$

Your program must produce exactly this total for exactly this input. It will be tested.

A note on money. Think carefully about how you represent EGP amounts. Currency and binary floating point have a long and unhappy history together. Whatever you choose, be ready to defend it.

Part C — How Data Must Be Organised

This section states *performance and behaviour* requirements. Choosing the data structures that satisfy them is your job, and each choice must be justified in your design document.

1. Looking up any restaurant, customer, rider, or order **by its identifier** must not require scanning every record.
 2. A restaurant's menu must be displayed in **the order the items were added**, consistently, on every run.
 3. The platform must be able to list **every distinct cuisine category** currently offered, with no duplicates.
 4. Restaurants must be listable **sorted by rating, highest first**, with ties broken alphabetically by name.
 5. Orders that are READY but not yet assigned must be held so that **the longest-waiting order is dispatched first** — except that orders from Gold customers jump ahead of all Bronze and Silver orders regardless of waiting time.
 6. Each customer's **last five searches** must be retrievable, newest first, without keeping the whole history.
 7. Two records describing the same real-world entity must never appear twice in any collection. Decide what "the same entity" means for each type and make your objects behave accordingly — including when they are used as keys or stored in sets.
 8. Any collection exposed outside the object that owns it must not be modifiable by the caller.
-

Part D — Reporting and Analytics

We need the following reports. Each must be produced from the live data using the modern Java toolkit — declarative pipelines over data, not hand-rolled accumulation loops.

1. Total revenue for a given date range.
2. The top five restaurants by revenue, for a given month.
3. Average order value per district.
4. Every restaurant that has an average rating above 4.5 **and** at least twenty completed orders.
5. A count of orders grouped by their current status.
6. Each rider's completed deliveries and their average delivery duration, sorted by deliveries descending.
7. The most frequently ordered menu item across the whole platform — noting that when no orders exist at all, there is no such item, and the report must express that absence honestly rather than inventing a value or crashing.
8. A customer's full order history, newest first, with the total they have spent.
9. The peak ordering hour of the day across all orders.
10. Every customer who has not ordered in the last thirty days.

Dates and times throughout the system must use the modern date-and-time API. Do not use the legacy classes.

You must also provide a search facility where the caller supplies the filtering condition — by district, by cuisine, by minimum rating, by price ceiling, or any combination — without the search code being modified for each new criterion.

Part F — When Things Go Wrong

The platform has failure conditions that Java's built-in exceptions do not describe: a restaurant that is closed, an item that is unavailable, a wallet without enough balance, an order transition that is not legal, a promotion that has expired or does not apply, a rider who is already busy, a stock shortage.

Design your own exception types for these. Organise them so that a caller can catch *one category* of platform failure without catching every unrelated problem in the JVM.

For each one you must decide: **checked or unchecked?** State the rule you applied, and apply it consistently.

Part G — Design Decisions You Must Make and Defend

You have studied design patterns. This section does **not** tell you which ones to use. It describes problems in this system whose shape should feel familiar. For each, choose an approach, implement it, and justify it in your design document — including what you rejected and why.

1. **Pricing varies by promotion type**, and new promotion types will be added regularly. The order-total code must not grow a chain of type checks or a switch that must be edited every time marketing invents something.
 2. **Building an order is a multi-step affair** — customer, restaurant, address, several line items, an optional promotion, optional delivery notes — and most of those steps are optional. A constructor with nine parameters, most of them null, is not acceptable.
 3. **Creating the right kind of menu item** from a data file that contains a type field should not scatter type-decision logic across the codebase.
 4. **When an order changes status, several unrelated things must happen**: the customer is notified, the rider dashboard updates, the audit log records it, and statistics are recalculated. Adding a fourth reaction later must not require modifying the order.
 5. **The platform configuration** — fee rates, pool size, file paths — must be loaded exactly once and be reachable from anywhere that needs it, without being passed through every constructor in the system. Note that this one is more contentious than it looks, and its interaction with Part E is a legitimate thing to discuss.
 6. **A rider's dispatch behaviour differs** for motorcycle, bicycle, and car (range, speed, and eligible order size). Adding an electric scooter later must not mean editing existing rider code.
-

Part H — The Console Application

A single interactive console application drives everything. Suggested structure:

```
=====
      MASR DELIVERY — Main Menu
=====
1. Customer
2. Restaurant
3. Rider
4. Admin & Reports
0. Exit
=====
Choose an option:
```

Each area contains its own operations. For each operation you must decide the exact prompts, but the behaviour below is required.

Customer area

Operation	Input	Process	Output
Browse restaurants	filter criteria (optional)	apply the filters; list open restaurants sorted by rating	numbered restaurant list with district, cuisines, rating
Search	free text	match against name and cuisine;	matching restaurants, or a clear “nothing found” message
View menu	restaurant number	show the menu in insertion order, marking unavailable items	grouped menu with prices in EGP
Place order	items, quantities, address, optional promo code	validate everything, price per Part B, reject atomically on any failure	order summary with full price breakdown, or a precise error
Pay from wallet	order reference	check balance, deduct, confirm	new balance, or an insufficient-funds message
Track order	order reference	look up current status	status and elapsed time since placement
Cancel order	order reference	verify the transition is legal, refund to wallet	confirmation, or the reason cancellation is not permitted
Order history	—	newest first with lifetime total spent	history listing and total

Restaurant area

Accept or reject a pending order; mark an order preparing, then ready; toggle item availability; add or remove menu items; adjust daily stock; view today’s orders and revenue.

Rider area

Go on or off duty; view the currently assigned order; mark it picked up; mark it delivered; view personal delivery statistics.

Admin area

Add or remove a restaurant; create a promotion; run any report from Part D; view platform-wide statistics.

Validation Rules

- Identifiers are unique within their type; duplicates are rejected.
 - Mobile numbers must be 11 digits and begin with 010, 011, 012, or 015.
 - Prices, quantities, and weights must be greater than zero.
 - Ratings are between 0.0 and 5.0 inclusive.
 - An order must contain at least one line item.
 - Orders cannot be placed with a closed restaurant, or for an unavailable item, or beyond available stock.
 - Wallet balance may never go negative.
 - Promotion codes are case-insensitive, must be unexpired, and at most one applies per order.
 - A promotion with a minimum-subtotal condition is rejected, with an explanation, when the subtotal is below it.
 - Only an available rider may be assigned an order.
 - Only legal status transitions are permitted.
 - A delivery address must belong to the customer placing the order.
 - Menu input that is not a valid option re-prompts rather than terminating.
-